

Homework 6

GWU CSCI 1112 - Spring 2026

April 9, 2026

1 Introduction

This homework is intended to reinforce your understanding of working with maps in general. Your task is to implement and test both a treemap and a hashmap, as well as to complete some functions that use their functionality.

1.1 Evaluation Date

April 17, 2026 at the beginning of your scheduled lab

1.2 Evaluation Method

Handwritten programming quiz where you will need to recount in pseudo-code aspects of this programming homework. The manual programming tasks will be relatively simple, but they will require a student to have practiced the homework to immediately understand and answer the question. Students who have not practiced or at least studied this homework, will generally be guessing during the quiz and they will typically require a long time to complete the quiz. Quizzes are timed and that time is capped at 30 minutes for all students including DSS students as this quiz involves recitation of already learned and practiced concepts and is not asking for full creative thinking. *Someone who has done this work should be able to finish the associated programming quiz in less than 10 minutes. If you have a handwriting disability, please talk to the professor before the evaluation date.*

1.3 Grading Rubric

I leave this here as an artifact for your reference. This is how this homework was graded when we evaluated student implementations. A rubric was provided with each homework and as the homeworks progressed, the percentages would encompass more critical metrics.

`UnitTests.java` AND `Analysis.java` must both compile otherwise the assignment will receive a zero grade.

Coding Exercise:

- 15% For binary tree implementation
- 15% For hash table implementation
- 20 % Passing all unit tests (10 % each block)
- 30 % For sufficiency of comments (10% for File Headers, 10% for Function Headers, 10% for Inline)
- 20 % Consistency of coding style

Analysis Exercise:

- 40% for analysis of binary-tree (balanced and unbalanced) operations
- 40% for analysis of hash table operations
- 20% for clarity of reasoning and quality of arguments

1.4 Comments

This section applies to graded student implementations. It is an artifact left for reference so you can understand expectations in the framework.

In this and future homeworks, you will have increased responsibility with commenting of code. Documentation is as critical a part of a programmer's job as programming code. In the previous frameworks, we have provided ample documentation for every file as a model for proper documentation procedures. It is your now responsibility to properly document your implementations in their entirety. This amount and type of documentation is an industry standard and we have provided sufficient examples in all of the assignments so far for you to base your work on.

In this and any future assignments, you will be expected to include the same types and detail for your documentation. There are three classes of comments:

- File header comments
- Function header comments
- Line comments

File header comments summarize the content of the file so that a programmer can quickly understand what is in the file without scanning through the file itself, provide clear natural language descriptions that can be matched by text based file search utilities, and associate authors, organizations, and copyrights with the code.

Function header comments document how to use functions in detail. The style used in this class is defined by the [Doxygen Standard](#) which facilitates the automatic generation of html web pages from code. All functions must have a comment that precedes the function declaration, *i.e.* must be before the function so that it clearly stands out, and this comment must describe 3 things when applicable: what the function does, every input into the function, and what the function returns if anything.

Line comments have a variety of roles. Typically they are used explain the internal logic within a class and often are associated with descriptions of the flow of execution inside a function. However, they are also associated with instance variables and explaining their role in the class, so they are not exclusively associated with functions. Line comments should be written in plain English and should add information that is not obvious from the code itself. Good line comments should help a reader understand the program even if that person does not understand the language or programming in general. Line comments are not necessary for every line of code, and can be used in groups to explain several steps grouped together. In general, line comments should be left justified to the same indentation as the current coding block and they should precede the upcoming code to prepare the reader for the code they are about to read.

1.5 Use of Imported Java Types

A goal of this class is to develop an understanding of how to build data structures. We will be developing our own classes for ArrayLists, LinkedLists, Stacks, Queues, BinaryTrees, etc. For this reason, you are not permitted to import any of the existing classes from any library unless explicitly stated.

2 Maps

As we discussing during class, a map is an abstract data type that associates, *i.e.* maps, keys to values. A map allows programmers to insert, retrieve, and delete values in the data structure quickly using a key. The key can also be a much more complex representation that is defined by the data in the value object rather than a simple, arbitrary integer value. To give a little more context, let's consider the [Dewey Decimal Classification](#).

Libraries traditionally index books within their collections using a proprietary organizational model originall pioneers through the [Dewey Decimal Classification](#). This classification system associates a key, *i.e.* an index consisting of numbers and letters, with a book to dictate where in a library that book is stored. This system basically defines how

to generate a key in order to prioritize library organization by topic and attempts to ensure that books with similar subject matter are more tightly clustered within the library over a more simple organizational model such as author and/or title. Consider how difficult it would be to use a library to cross-reference computer science topics if books by Eberly and Sipser were stored on different floors or even different buildings rather than within several shelves of one another. The key generated by Dewey classification directs library users directly to a floor, aisle, and shelf within the library and allows library users to quickly filter out thousands or even millions of non-relevant books from a search.

The library organization model represents a map which facilitates quick access to information. Programmers need to be able to apply similar systems to ensure that search efficiency is optimal. We have also noted that there is close relationship between search efficiency and sorting where our most efficient search algorithm in “flat” data, *i.e.* binary search, requires an expensive sort operation to impose the organization necessary to gain the advantage of a fast search. To make our systems as efficient as possible, we need to reduce the cost of insertion and maintain order so that the combination of sort and search is efficient. In order to achieve this requirement, we will look at map implementations using both binary trees and hash tables.

As we noted in class, the binary tree is a hierarchical data structure whose organizational structure produces the same search efficiency as binary search as long as the tree is balanced. The expensive aspect of maintaining the binary tree is due to rebalancing the tree which may be performed in response to insertion. If rebalancing is as efficient as optimal quicksort, then a binary tree is no less efficient than a combination of quicksort and binary search and if rebalancing is more efficient than optimal quicksort then we can use a binary search approach that is more efficient overall than one dependent on quicksort.

We also looked at hash tables and noticed that insertion into the hash table may occur in constant time as long as search conditions are optimal; however, the efficiency of search in a hash table is subject to a number of factors, primarily the bias of the hashing algorithm and the ratio between the number of elements stored to the number of buckets. If the hash algorithm results in the same index being selected, then we are effectively storing data in a single linked-list and our search efficiency will be no better than searching a linked-list, and even if the hash algorithm is unbiased, when the ratio of elements to buckets is large then every bucket contains a lengthy linked-list.

In this homework, we will implement maps using a binary tree and a hash table. We will then run a few experiments to confirm experimentally whether the performance of each of these maps approaches the estimates derived through Big O analysis. Both implementations will subscribe to the map interface.

2.1 Framework

A framework is provided that consists of a number of completed classes and a few classes that you will need to implement. The framework consists of the following files:

Analysis.java	ListNode.java	MapReader.java	UnitTests.java
HashMap.java	mapdata	TreeMap.java	Utilities.java
KVP.java	Map.java	TreeNode.java	

You must implement functions in `TreeMap.java` and `HashMap.java` which are tested and used by the executable files `UnitTests.java` and `Analysis.java`. To facilitate implementation of your maps, a `TreeNode` class is provided in `TreeNode.java` for your binary tree and a `ListNode` class is provided in `ListNode.java`. You will also need to familiarize yourself with the key-value pair implementation in `KVP.java`.

Copies of the original files provided are located in `HashMap_Framework.java` and `TreeMap_Framework.java`. An example solution for the hash map is provided in `HashMap_Solution.java` and a recursive Tree Map implementation is provided in `TreeMap_RecursiveSolution.java` and an iterative implementation is provided in `TreeMap_IterativeSolution.java`

As far as the remaining files, `mapdata` contains sample data that maps universities to their location, `MapReader.java` loads files structured like `mapdata` from disk into memory, `Map.java` defines the abstraction layer met by both implementations, and `Utilities.java` provides support functions.

2.1.1 profile

A `profile` is an array that is used to track the performance of an operation.

Recall that in Homework 2, we used a similar structure to experimentally quantify the number of comparisons needed to perform sorting operations. For this homework, we will use a similar technique to quantify the number of comparisons performed for map operations.

For this case, our profile array will be structured such that it consists of one element which will be initialized to zero and will be incremented each time a comparison is performed inside the map during either the `get` or `set` operation. This array must be allocated before calling either operation and is passed into the operation as a parameter.

It may be easy to overcount the number of comparisons; however, recall that we can typically perform a comparison and store the result so that we do not need to perform a comparison multiple times. We can also use methods that produce non-binary results, for example the `compareTo` function returns three possible values, -1 if the parameter precedes the operand, 0 if the parameter is equivalent to the operand, and 1 if the parameter follows the operand.

The `compareTo` function is illustrated in Module 10 in the `BinaryTreeString` example and the example in the Appendix in Section 5 at the end of this document updates the `BinaryTreeString` example and illustrates how to use the `compareTo` function combined with the `profile` field to count comparisons.

3 Requirements

You must implement the following methods in both the `TreeMap` class and the `HashMap` class using the following function signatures:

```
public void put(String key, String value, int[] profile);
public String get(String key, int[] profile);
public boolean delete(String key, int[] profile);
public void clear();
```

Please refer to the Section 2.1.1 above for detailed information on the `profile` parameter.

3.1 Classes

This section defines specific requirements for each of the Map implementation classes.

3.1.1 TreeMap

You must implement the map interface methods in the `TreeMap` class. Each element in the tree should be maintained by an instance of a `TreeNode` which provides fields for the key-value pair associated with a tree element and references for potential left and right elements that may be children for a given binary tree element.

The constructor for the class takes a `rebalance` flag that specifies whether or not the tree should be rebalanced after every insertion. If the flag is true, then the provided `balance` function must be called after every insertion is completed, and if the flag is false, then balance should not be called. Our balancing policy and our balancing algorithm are suboptimal brute force approaches, and in a real-world implementation, we would want to use a more sophisticated algorithm like AVL or red-black to meet the expected logarithmic time efficiency. We will not count the operations in the rebalancing process in terms of efficiency, but you should keep in mind that balancing adds a cost and high frequency balancing will affect the overall performance of insertion into the tree; however, if balancing is not performed frequently enough, then the overall performance of search will be affected.

Note that node deletion is actually performed in the rebalance operation. Each `TreeNode` contains a `delete` flag. When this flag is set to `true`, the rebalance operation ignores any node with the `delete` flag set which implicitly deletes it through garbage collection. Balancing can be done on a subtree given the root of the subtree, so `delete` should call `balance` on the node to be deleted even if the rebalance flag is disabled in order to ensure the specified node is actually removed from the map. This may be a little redundant as this generates two rebalancing operations per delete for a balanced binary tree; however, intentionally implementing a non-rebalancing, ordered tree is unrealistic in a real-world scenario as this would undermine the benefits of choosing a tree in the first place. Chalk this up to a contrived scenario for academic purposes to show why we generally want a self-balancing tree for efficiency. We could eliminate this redundancy by not balancing in `delete`; however, since we can turn on and off tree rebalancing in general, not locally rebalancing the subtree in `delete` would mean that nodes marked for deletion in a non-rebalancing tree would never actually be deleted, so again this is a weird concession for academic purposes.

3.1.2 HashMap

You must implement the `map` interface methods in the `HashMap` class. Each element in the hash table should be maintained by an instance of a `ListNode` which provides fields for the key-value pair associated with a hash table element and a reference for any potential next element that may follow a given hash table element in one of the lists.

The constructor for the class takes the number of buckets in the array as a parameter. You will find that ratio of the number of buckets in the array to the number of elements stored in the entire hash table is a key that heavily influences the efficiency of map operations in the hash table. Recall that a hash table is an array of linked-lists and the `hash` function is used to compute the index of the linked-list in the array where a key must be stored. When getting or setting, you must hash the key using the `hash` function to compute the index of the bucket in which the key-value pair should be stored.

3.1.3 Map Reminders

Recall that each key must be unique.

This means that upon `get` and once the bucket is identified through hashing, for example, you must perform a search on the key in the associated linked-list to determine whether the key is already stored in the linked-list before performing any insertion. This means that we cannot arbitrarily perform tail insertions on the linked-list, so there is no benefit of storing a tail pointer because we will end up searching for the tail anyway.

For the sake of profiling comparisons, only count the comparisons when searching the linked-list itself.

3.2 Functions

This section gives a general explanation of the requirements for each `Map` function associated with the `Map` interface.

`void put(String key, String value, int[] profile)`

Recall that keys must remain unique in a map so there can only be one key-value pair with a given key. Therefore, `put` is itself a search operation. If the key does not exist in the map, then `put` allocates a new key-value pair and inserts that key-value pair into the map at the appropriate location; however, if the key already exists in the map, then `put` instead locates the key-value pair and updates the value associated with that key.

Please refer to the Section 2.1.1 above for detailed information on the `profile` parameter.

String get(String key, int[] profile)

`get` searches through the map for the specified key. If the key is found, `get` returns the associated value. If the key is not found, `get` returns a `null` value to indicate that the key does not exist in the map.

Please refer to the Section 2.1.1 above for detailed information on the `profile` parameter.

String delete(String key, int[] profile)

`delete` searches through the map for the specified key. If the key is found, `delete` removes the key value pair from the map. If the key is not found, `delete` does nothing. `delete` returns a boolean indicator to report whether or not the key value was found.

Please refer to the Section 2.1.1 above for detailed information on the `profile` parameter.

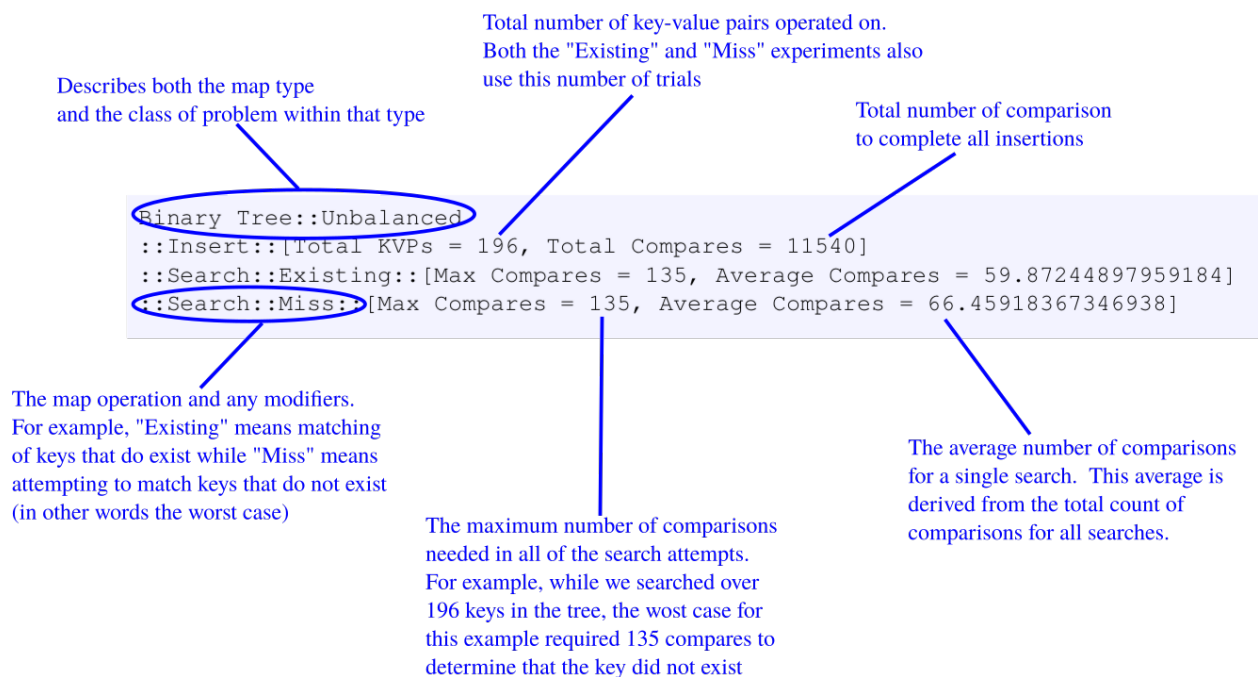
void clear()

`clear` resets the map such that the map is empty after the clear operation is completed. This operation can leverage the Java garbage collector so the implementations can typically be very simple.

4 Analysis

Again, this section is provide for reference so that you understand the entire context of the assignment and what students are generally expected to do in this homework. Even though if is not graded, it is worth your time to perform this analysis.

Through analysis you will argue whether the experimental data gathered from these map implementations supports our Big O analyses. The experimental data is structured as follows:



4.1 Data

```
Binary Tree Map::Unbalanced
::Insert::[Total KVPs = 199, Total Compares = 1447]
::Search::Existing::[Max Compares = 16, Average Compares = 8.266331658291458]
::Search::Miss::[Max Compares = 11, Average Compares = 8.341708542713567]

Binary Tree Map::Balanced
::Insert::[Total KVPs = 199, Total Compares = 1253]
::Search::Existing::[Max Compares = 8, Average Compares = 6.758793969849246]
::Search::Miss::[Max Compares = 8, Average Compares = 7.608040201005025]

Hash Table Map::10 Buckets
::Insert::[Total KVPs = 199, Total Compares = 2048]
::Search::Existing::[Max Compares = 29, Average Compares = 11.241206030150753]
::Search::Miss::[Max Compares = 29, Average Compares = 20.35175879396985]

Hash Table Map::100 Buckets
::Insert::[Total KVPs = 199, Total Compares = 283]
::Search::Existing::[Max Compares = 7, Average Compares = 1.9748743718592965]
::Search::Miss::[Max Compares = 7, Average Compares = 2.0351758793969847]

Hash Table Map::199 Buckets
::Insert::[Total KVPs = 199, Total Compares = 233]
::Search::Existing::[Max Compares = 6, Average Compares = 1.5376884422110553]
::Search::Miss::[Max Compares = 6, Average Compares = 0.9949748743718593]

Hash Table Map::1000 Buckets
::Insert::[Total KVPs = 199, Total Compares = 201]
::Search::Existing::[Max Compares = 3, Average Compares = 1.1105527638190955]
::Search::Miss::[Max Compares = 1, Average Compares = 0.18090452261306533]
```

4.2 Report

You will submit a report in pdf format using the same root naming convention defined for the zip file in Section ?? that discusses the computational efficiency for get and set operations.

Remember that a Big Oh analysis consists of three parts and your analysis is incomplete without fulfilling all three parts: 1) define the size of the problem and the associated variables that represent these sizes, 2) analyze the computational efficiency of the algorithm as accurately as necessary in terms of 1, and 3) summarize the overall efficiency in terms of Big Oh notation.

Your statistics may vary somewhat from the above sample depending on the way you implement your sorting algorithm; however, the general order of magnitudes for each of the statistics should be roughly the same. If your numbers are wildly different than the sample statistics, analyze the sample statistics in your pdf rather than your own data.

4.2.1 Binary Tree

Discuss computational efficiency for both map operations, get and set, in a binary tree using Big O analysis depending on whether the binary tree is balanced or unbalanced. You should develop an objective, analytical hypothesis of the efficiency of this structure based on the length of a path through the tree and then use the experimental data above to support your Big O analysis.

4.2.2 Hash Table

Discuss computational efficiency for both map operations, get and set, in a hash map using Big O analysis depending on the ratio between the number of buckets M with respect to the number of elements n in the data set. You should develop an objective, analytical hypothesis of the efficiency of this structure based on the statistical average distribution of elements and then use the experimental data above to support your Big O analysis.

5 Appendix

The following code is an updated example of the `BinaryTreeString` example from Module 10 and illustrates how to use the `compareTo` function to lexicographically compare strings and the `profile` field to count comparisons.

```
public class BinaryTreeString {

    public void add (String data) {
        // ...
    }

    public boolean contains (String str, int[] profile) {
        if (numItems == 0) {
            return false;
        }

        return recursiveSearch (root, str, profile);
    }

    boolean recursiveSearch (TreeNode node, String str, int[] profile) {
        // Here we call compareTo and store the result. By storing the
        // the result, we only need to perform the comparison once.
        // compareTo returns three values, negative, zero, and positive,
        // which represent before, same as, and after so any branching on
        // may be more sophisticated than what could be accomplished with
        // just one if...else branch.
        int result = str.compareTo (node.data);
        profile[0]++; // count only a single comparison

        if ( result == 0 ) {
            // Found.
            return true;
        }

        // Otherwise, navigate further.
        if ( result < 0 ) {

            // Go left if possible, otherwise it's not in the tree.
            if (node.left == null) {
                return false;
            } else {
                return recursiveSearch (node.left, str, profile);
            }

        } else {
            // Go right if possible.

            // ... similar to above ...
        }
    }
}
```